

ACTIVIDAD 4: MANIPULACIÓN DE CADENAS Y PUNTEROS EN C/C++ - LPR 5° 3° A-B (Presencial/Virtual Asincrónico - Actividades de Contingencia y Continuidad Pedagógica) 2026

De Orvion Robotics

Colegio: Técnica 1 de Vicente Lopez

Profesores:

- Mansilla Muñoz York Elías
- Yamil Ganduglia

Url del repositorio: <https://github.com/teolaguna/CADENASPUNTEROS.git>

Actividades:

CUESTIONARIO TÉCNICO DE CONTROL Respondan las siguientes 3 preguntas dentro del informe PDF final que entregarán en Moodle.

 Pregunta 1: Diferencia entre & y *

- Explicación: El operador ampersand (&edad) obtiene la ubicación física en la RAM, mientras que el asterisco (*p) accede al contenido de esa dirección.
- Pregunta: ¿Qué valor imprime en pantalla la instrucción `cout << p;` comparada con `cout << *p;`? 

Pregunta 2: Error Crítico al Guardar Entrada

- Explicación: Intentar escribir `cin >> p;` sin el asterisco es un error grave de compilación porque se intentaría sobrescribir la dirección de memoria física asignada al puntero.
- Pregunta: ¿Por qué es obligatorio desreferenciar el puntero (*p) al usar `cin` para modificar el valor de la variable original?

 Pregunta 3: Formato Hexadecimal

- Explicación: Las direcciones de memoria RAM se muestran en formato hexadecimal (base 16) anteceditas por 0x (ej: 0x61fe1c).
- Pregunta: ¿Por qué el sistema operativo utiliza la notación hexadecimal en lugar de decimal para representar las direcciones de memoria?

Respuestas:

1. `cout << p;` vs `cout << *p;` `p == &edad` imprime la dirección hexadecimal (ej 0x2f2d5ff844), el casillero en RAM. `*p == edad` imprime el valor (ej 20). Sin * ves dónde está guardado, con * ves qué contiene.
2. ¿Por qué `cin >> *p;` obligatorio? `p` es `int*` (dirección), `*p` es `int&` (referencia al valor). `cin >> p` no tiene sobrecarga para `int*` y si compilara sobrescribiría la dirección, rompiendo el

puntero -> comportamiento indefinido. *p desreferencia y escribe directo en la casilla original de edad (equivalente a cin >> edad).

3. ¿Por qué hexadecimal? 1 dígito hex = 4 bits -> más compacto (32 bits = 8 hex vs 10 decimales), mapeo 1:1 con binario para máscaras/offsets, y límites de página visibles (0x1000 = 4096). En decimal se pierde esa legibilidad de bajo nivel.

Figura 1

```
=====
CONTROL DE ACCESO RAM - ESTUDIANTE: Teo Laguna
=====
=> Ingrese su edad: 20

--- ANALIZANDO ACCESO SEGURO EN MEMORIA ---
[ACCESO APROBADO] El usuario es mayor de edad.
Edad registrada: 20 anos.
Direccion fisica en RAM Hexadecimal: 0x2b8fffb94
=====
```

figura 2:

